

SatQuery AI — Core Java & OOP Execution Plan

SIH 2026 Project Blueprint

Problem Statement ID: SIH26167

Project: SatQuery AI

Primary engineering strength: Core Java and Object-Oriented Programming

Document purpose: Build, validate, demonstrate, and package a reliable natural-language satellite-imagery analysis system without making Python or Spring Boot a team dependency.

Winning principle: A reliable, clearly demonstrated system is stronger than an ambitious system that the team cannot explain, test, or operate during judging.

1. Executive Summary

SatQuery AI is a natural-language satellite-intelligence assistant. A user uploads one image, two images from different dates, or an optical-SAR pair and asks a question in plain English. SatQuery validates the input, identifies the task, selects the appropriate specialist analysis module, returns an answer with visual evidence, and preserves a reproducible execution trace.

The team's responsibility is the **reliable Java/OOP application backbone**. Core Java will own the request model, validation, controller, tool interfaces, routing patterns, execution status, report generation, error handling, testing, and application workflow. AI models will be treated as replaceable engines behind a stable integration boundary. The team does not need to train models in Python or build the project on Spring Boot.

The first milestone is not fine-tuning. The first milestone is a complete vertical slice:

Plain Text

Upload image → Ask question → Validate → Select tool → Receive result
→ Show evidence and trace → Export report

The development order is baseline-first, but remote-sensing adaptation is a mandatory final deliverable. By Week 1, the team must select one adaptation route using BigEarthNet.txt or approved open-source training data. The adapted component must be

evaluated on held-out examples and integrated through the Java ModelClient boundary. Fine-tuning implementation is optional; adaptation evidence is mandatory.

2. Final Outcome Required

At final evaluation, a non-GIS user must be able to upload satellite imagery, ask a natural-language question, receive a useful result, see the selected analysis path, inspect evidence and limitations, and export a report.

The exact five mandatory capabilities from the official SIH problem statement must be copied into the project workspace and converted into the acceptance matrix below. Do not rely only on a paraphrase.

Definition of done

Area	Completion standard	Evidence to prepare
Official requirements	Every official requirement maps to a working SatQuery feature	Requirement matrix and demo query
End-to-end flow	Upload → query → routing → tool → answer → evidence → report works repeatedly	Screen recording and live rehearsal
Java controller	Controller validates requests and dispatches specialist modules through interfaces	Source code, class diagram, unit tests
OOP architecture	Factory, Strategy, and Observer responsibilities are clear and testable	Architecture diagram and code walkthrough
Satellite analysis	Single-image questions, change analysis, and optical–SAR reasoning are demonstrated according to actual scope	Sample outputs and limitations
Explainability	Selected task, tool, parameters, status, confidence, and limitations are visible	Trace panel and JSON trace
Evaluation	Held-out examples are compared with a simple baseline	Evaluation table and error analysis

User experience	A non-technical user can operate the system without specialist training	Usability rehearsal and screenshots
Reliability	Invalid input, missing modalities, timeouts, and unavailable engines are handled gracefully	Failure-case test log
Packaging	Code, setup instructions, test data, screenshots, demo video, and presentation are complete	Release checklist

3. Product Definition

3.1 Primary user

Use one primary user consistently in the presentation: a disaster-management or public-sector analyst who needs rapid answers from satellite imagery without advanced GIS expertise. Secondary users may include agriculture officers, urban planners, environmental-monitoring teams, and researchers.

3.2 Core user promise

SatQuery AI lets a user ask questions about satellite images in plain English and receive an explainable answer supported by the correct remote-sensing analysis tool.

3.3 Primary user workflow

1. The user uploads one image, a pair from different dates, or an optical–SAR pair.
2. The user enters a natural-language question.
3. Java validation checks file count, format, modality, dates, and query compatibility.
4. The Java controller classifies the task using rules or a replaceable classifier endpoint.
5. A Factory creates the appropriate task handler.
6. A Strategy object selects the specialist execution path.
7. The selected engine returns an answer, visualization, confidence information, and metadata.
8. An EvidenceCombiner assembles the output.

9. An Observer publishes progress to the trace panel.
10. ReportBuilder exports a PDF and JSON record.

3.4 Demonstration queries

Capability	Demonstration question	Expected visible output
Single-image VQA	“What type of land use is visible in this image?”	Answer, image context, confidence/limitations
Grounding or captioning	“Where are the buildings or roads located?”	Scene description or localized evidence
Change analysis	“What changed between these two dates?”	Change map, explanation, dates, trace
Optical–SAR reasoning	“What evidence from both sensors indicates flooding?”	Optical evidence, SAR evidence, combined interpretation
Report export	“Export this analysis.”	PDF and JSON with trace and metadata

4. Team Ownership and Skills Fit

Every workstream must have one accountable owner and one backup owner. “The team” is not an owner. Replace placeholders with real names during the first planning meeting.

Workstream	Primary owner	Backup owner	Java/OOP role
Product and SIH requirements	Team lead	All members	Acceptance matrix and demo priorities
Core domain model	Java developer	Team lead	QueryRequest, ImageAsset, TaskResult, Evidence, Report
Controller and routing	Java/OOP developer	Java developer	Factory, Strategy, Observer, dispatch logic

File and input validation	Java developer	Frontend member	Format, modality, count, date, and error checks
User interface	UI developer	Java developer	Calls the Java application interface and displays trace
AI integration boundary	Assigned integration owner	Java developer	Java HTTP Client or Java-compatible inference runtime
Change-analysis module	Analysis owner	Java developer	Contract, inputs, outputs, and visualization handoff
Optical-SAR module	Analysis owner	Team lead	Fusion contract and evidence presentation
SQL/history	Java or SQL owner	Java developer	Query history, traces, report metadata
Evaluation	Team lead	All members	Test library, baseline, metrics, error analysis
Presentation and demo	Team lead	UI developer	Six-slide deck, script, live flow, fallback video
Packaging	Java developer	Team lead	Build instructions, test data, release archive

Team working method

The team should hold one short daily checkpoint during active development. Each member states what was completed, what is blocked, and what will be demonstrated next. Every feature is merged only after a reviewer can run it from a clean environment.

5. Java/OOP Architecture

5.1 System architecture

Plain Text
User Interface

```

Java Application Layer
|-- Request and domain objects
|-- File and modality validation
|-- AgentController
|-- HandlerFactory
|-- Strategy selection
|-- SatelliteTask interface
|-- TraceObserver and status events
|-- EvidenceCombiner
|-- ReportBuilder
|-- Query history and metadata
|
AI Integration Boundary
|-- Java HTTP Client to a model endpoint, OR
|-- Java-compatible local inference runtime
|-- Stable request/response contract
|
Specialist Engines
|-- Single-image VQA
|-- Grounding or captioning
|-- Change detection / change VQA
|-- Optical-SAR fusion
|
Persistence and Files
|-- SQL database or embedded database
|-- Uploaded imagery
|-- Change overlays
|-- Exported PDF and JSON reports

```

The diagram intentionally separates **what the team owns** from **what is an external or replaceable AI engine**. The Java application remains testable even when an AI engine is unavailable.

5.2 Core domain objects

Class or interface	Responsibility
QueryRequest	Stores the user question, files, dates, modalities, and project context
ImageAsset	Stores file identifier, modality, acquisition date, dimensions, and validation state
TaskType	Enum for VQA, GROUNDING, CHANGE_ANALYSIS, FUSION, and REPORT

SatelliteTask	Common contract implemented by every specialist handler
TaskHandler	Executes one task using the selected strategy and engine client
TaskResult	Standard result with answer, evidence, confidence, limitations, and metadata
TraceEvent	One observable step in the execution history
Evidence	Image overlay, change map, textual support, or sensor-specific explanation
Report	Structured export model for PDF and JSON output
ModelClient	Stable adapter to a remote endpoint or Java-compatible runtime

5.3 Design patterns

Factory Pattern: HandlerFactory creates a handler for the selected task. The controller does not contain large chains of hard-coded construction logic.

Strategy Pattern: VqaStrategy, ChangeStrategy, and FusionStrategy implement task-specific behavior behind a common interface. The controller can select a strategy without changing the user interface.

Observer Pattern: TraceObserver receives status events such as REQUEST_RECEIVED, INPUT_VALIDATED, HANDLER_SELECTED, ENGINE_STARTED, EVIDENCE_READY, and REPORT_GENERATED.

Adapter Pattern: ModelClient hides whether an engine is remote, local, deterministic, or a future replacement. The rest of the Java application receives the same response contract.

5.4 Tool contract

Every registered tool must define the following contract before implementation:

Field	Required content
Tool name	Unique human-readable identifier
Purpose	Capability provided
Accepted inputs	Image count, modalities, dates, and file types

Parameters	Thresholds, bands, dates, or other controls
Output	Answer, visualization, confidence information, and metadata
Failure response	Clear reason and user correction
Test cases	At least three normal cases and two failure cases

5.5 Example Java-style contract

Java

```

public interface SatelliteTask {
    TaskType getTaskType();
    TaskResult execute(QueryRequest request);
    ValidationResult validate(QueryRequest request);
}

```

The exact implementation may change, but the principle must remain: each tool is independently replaceable, testable, and observable.

6. Model and Data Strategy

6.1 Baseline-first approach

Begin with one stable pretrained remote-sensing model or model endpoint for the initial single-image workflow, plus a deterministic image-processing baseline for change analysis. Do not begin by training multiple models.

GeoChat or an RS vision-language model may be evaluated as a single-image baseline. EarthGPT or a comparable multimodal approach may be evaluated for optical–SAR reasoning. The team must choose one primary path after a short feasibility test and must not multi-track indefinitely.

6.2 AI integration without Python ownership

The team can integrate AI without making Python part of its core responsibility. Use the following controlled Java-centered boundaries:

Option	How Java interacts	When to use
--------	--------------------	-------------

Core Java HTTP application	Java built-in HttpServer receives requests and returns JSON	Primary application path; no Spring Boot required
Model integration	Java HttpClient sends JSON to a documented adapted-model endpoint	Primary AI path; keep the request/response contract stable
Java-compatible runtime	Java calls ONNX or another compatible runtime	Secondary option only after a small feasibility test
Deterministic baseline	Java invokes packaged comparison logic or local JSON fixtures	Required fallback for change visualization and offline demo

The team should not claim to have trained or implemented a model unless it has actually done so. The presentation should say that Java orchestrates replaceable AI engines through a defined interface.

6.3 Data strategy

Use a manageable subset of paired optical and SAR imagery for development. Maintain separate development, validation, and held-out test partitions. Do not use test images during prompt tuning, threshold selection, or model choice.

Create a test library of 15–20 representative queries covering every mandatory capability. Each item should contain the input files, expected task label, expected evidence, acceptable answer characteristics, and evaluation result.

6.4 Data-quality checks

Before evaluation, confirm that optical and SAR pairs correspond to the same geographic area, bi-temporal images have clear acquisition dates, image dimensions and formats are valid, normalization is consistent, and the test set is held out. Clearly label any synthetic or manually annotated examples.

6.5 Fine-tuning rule

Remote-sensing adaptation is mandatory even though large-model fine-tuning is optional. Run a small adaptation experiment after the baseline and Java vertical slice work. Acceptable routes include a documented fine-tuning run, adapter/LoRA update, classifier head, prompt/domain adaptation, or an adapted open-source component. Continue only when the adaptation is reproducible and evaluated on held-out examples. If large-model fine-tuning is unstable, use a smaller defensible adaptation while freezing the baseline for

the live demo. The final package must contain the component name, data source, method, split, before/after result, and Java integration instructions.

7. Development Phases and Milestones

Phase 0 — Official requirement lock

Duration: One working day.

Copy the official SIH problem statement into the workspace. Extract the five mandatory requirements and create this matrix:

Official requirement	SatQuery feature	Evidence	Demo query	Owner	Status
Requirement 1	Feature name	Screenshot/output/test	Query	Name	Not started
Requirement 2	Feature name	Screenshot/output/test	Query	Name	Not started
Requirement 3	Feature name	Screenshot/output/test	Query	Name	Not started
Requirement 4	Feature name	Screenshot/output/test	Query	Name	Not started
Requirement 5	Feature name	Screenshot/output/test	Query	Name	Not started

Gate: No feature is complete without evidence in this matrix.

Phase 1 — Core Java feasibility

Duration: Week 1.

Create the repository, coding conventions, Java project structure, test folders, input folders, sample imagery, domain classes, interfaces, and a command-line or simple local UI entry point.

Implement `QueryRequest` , `ImageAsset` , `TaskResult` , `SatelliteTask` , `AgentController` , and a mock `SatelliteTask` . Verify Factory, Strategy, and Observer behavior with unit tests.

Deliverable: A Java program that accepts an image reference and question, validates the request, selects a mock handler, returns structured output, and records a trace.

Gate: If the AI engine is not available, continue with the mock and deterministic baseline. Do not block architecture work.

Phase 2 — Input, data, and evaluation foundation

Duration: Weeks 1–2.

Implement image loading, format validation, modality identification, date checks, pair validation, metadata extraction, and dataset partitioning. Create the 15–20-query test library and evaluation record format.

Record task classification, output quality, visualization availability, response time, confidence information, and failure type. Compare with a simple baseline.

Deliverable: A repeatable evaluation run on held-out examples.

Gate: No performance claim without a baseline and documented sample count.

Phase 3 — End-to-end vertical slice

Duration: Weeks 2–3.

Connect the UI to the Java application flow. Implement one-image upload, natural-language query entry, validation, routing, one VQA path, structured output, visible trace, and basic history.

Deliverable: A five-minute clean-environment demo.

Gate: This flow must work before advanced model integration or fusion expansion.

Phase 4 — Capability expansion

Duration: Weeks 3–5.

Add capabilities in this order:

1. Single-image VQA.
2. Grounding or captioning, depending on official requirements.
3. Bi-temporal change detection and change-oriented question answering.
4. Optical–SAR evidence combination.
5. Rich overlays and PDF/JSON report export.

For each capability define the input contract, Java handler, engine adapter, output evidence, confidence behavior, normal cases, and failure cases.

Deliverable: Every mandatory capability is accessible through the Java task registry.

Phase 5 — Controller and trace hardening

Duration: Weeks 4–5.

Implement intent selection, compatibility checks, handler dispatch, output combination, confidence handling, and trace persistence.

A trace record should contain:

JSON

```
{
  "query": "user question",
  "inputFiles": ["file identifiers"],
  "modalities": ["optical", "sar"],
  "selectedTask": "change_analysis",
  "selectedTools": ["change_handler", "model_client"],
  "parameters": {},
  "executionStatus": "success",
  "confidence": null,
  "limitations": [],
  "latencyMs": null
}
```

Use `null` or an unavailable state when confidence or latency has not actually been measured. Never invent values.

Phase 6 — UI, persistence, and report integration

Duration: Weeks 3–6.

The UI must provide upload, query entry, input preview, result display, overlays, trace expansion, confidence/limitation display, report export, and history access.

Use the Java application layer for validation, routing, integration, history, and report generation. Use a simple database or embedded SQL database only if the team can support it; otherwise use a well-structured local data store for the prototype. Keep model communication behind `ModelClient`.

Gate: Test the full flow from a clean browser or local UI session and a clean Java runtime environment.

Phase 7 — Evaluation and error analysis

Duration: Week 6.

Run the complete test library and categorize failures:

Failure category	Example	Required response
------------------	---------	-------------------

Wrong routing	VQA sent to change handler	Improve rules, labels, or routing strategy
Wrong answer	Scene misinterpreted	Improve prompt, engine, or fallback
Missing evidence	Answer has no supporting overlay	Add evidence or state limitation
Input error	Wrong file count or modality	Improve Java validation message
Slow response	Timeout or excessive latency	Resize, cache, simplify, or use fallback
False confidence	Incorrect answer shown as certain	Show uncertainty and require verification

Create a final table showing the baseline, SatQuery result, sample count, response-time observations, and known limitations. Use honest results.

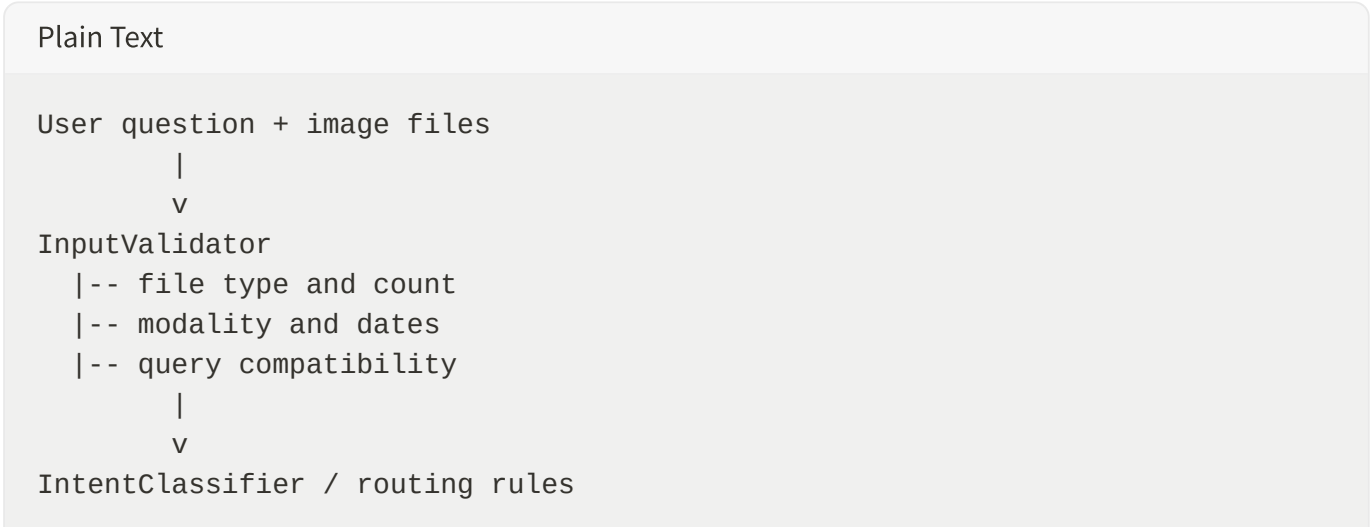
Phase 8 — Demo and packaging freeze

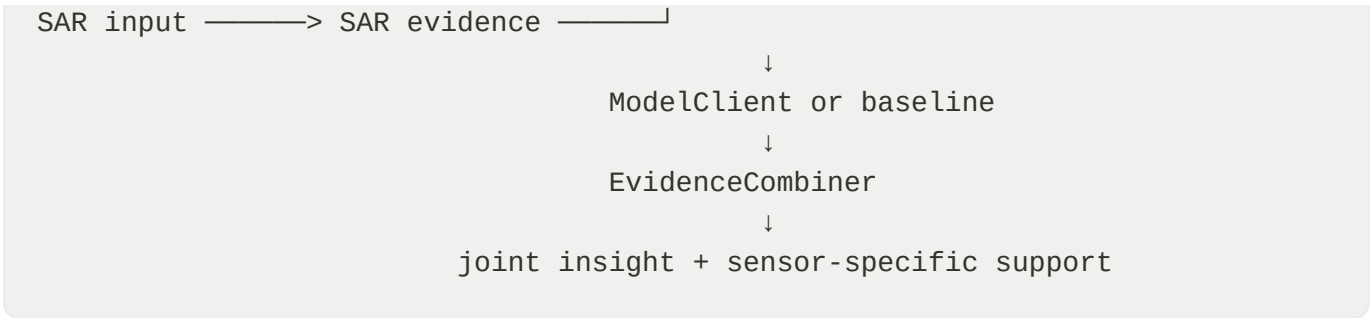
Duration: Final working days.

Freeze the code only after a clean-environment rehearsal. Prepare the live sequence, a five-minute recorded fallback, bundled test files, screenshots, setup instructions, the six-slide presentation, and team speaking roles.

8. Detailed Workflows

8.1 Agentic routing workflow





The UI must distinguish what came from optical evidence, what came from SAR evidence, and what is inferred from the combination.

8.4 Report workflow

Plain Text

TaskResult + TraceEvents + Evidence + Metadata

- ReportBuilder
- PDF report + JSON record
- history entry and downloadable output

9. User Interface Requirements

The interface should look like an operational console, not a generic chatbot. It must contain:

Area	Required content
Upload panel	One image, two-date pair, or optical–SAR pair
Query box	Plain-English question with example prompts
Input preview	File names, modality, date, and validation state
Result area	Answer, evidence overlay, change map, or sensor comparison
Trace panel	Step-by-step Java events and selected handler
Confidence area	Measured confidence or an honest unavailable/needs-review state
Report controls	PDF and JSON export
History	Previous queries and result status

Failure message

Clear action for invalid or incompatible input

10. Testing Strategy

10.1 Unit tests

Test validators, domain objects, Factory creation, Strategy selection, Observer notifications, error mapping, report serialization, and ModelClient response parsing.

10.2 Integration tests

Test upload-to-controller flow, controller-to-handler flow, handler-to-engine adapter flow, trace persistence, report creation, and fallback behavior.

10.3 Acceptance tests

For every official SIH requirement, prepare a normal case, a failure case, a screenshot, a result artifact, and a responsible team member.

10.4 Demo rehearsal tests

Perform at least three complete rehearsals from a clean environment. Test the live path, the offline/bundled path, and the recorded fallback path.

11. Risk Register and Fallbacks

Risk	Early warning	Primary response	Fallback
AI endpoint unavailable	No stable response in Week 1	Use mock contract and deterministic baseline	Bundled local demo results
Model quality weak	Incorrect answers on held-out cases	Improve prompt, routing, or evidence	Narrow the supported scope honestly
Fusion unstable	Contradictory optical/SAR output	Show separate evidence branches	Transparent comparison without joint claim
Integration complexity	Java flow blocked by engine details	Freeze adapter contract	Use recorded or precomputed engine response

UI delay	Long live response	Cache, resize, asynchronous status	Preloaded demo data
Incorrect routing	Wrong handler selected	Improve routing rules and tests	Manual task selector for demo with disclosure
False confidence	Wrong result looks certain	Display limitations and review state	Remove unsupported confidence score
Team skill mismatch	New framework blocks progress	Return to Core Java and simple interfaces	Keep infrastructure minimal
Demo failure	Network, file, or hardware issue	Rehearse and package locally	Recorded backup video

Stop rules

Stop fine-tuning if the baseline works and the fine-tuned result does not improve held-out evaluation. Stop adding frameworks if they slow the team or cannot be defended. Stop adding features when the six-slide demo flow is not reliable. A smaller honest product is preferable to a large unstable one.

12. Execution Timeline

Time	Goal	Required output	Gate
Day 1	Official requirement lock	Five-requirement acceptance matrix	Every requirement has an owner
Week 1	Core Java foundation	Domain objects, interfaces, controller, mock tool, tests	Java flow runs locally
Week 2	Validation and evaluation	Input checks, test library, baseline comparison	Held-out run is repeatable
Week 3	Vertical slice	UI → Java controller → result → trace	Five-minute clean demo
Week 4	Capability expansion	Change and grounding handlers	Each has contract and tests

Week 5	Fusion and reporting	Optical–SAR workflow, overlays, PDF/JSON	Full mandatory path is visible
Week 6	Hardening	Error handling, packaging, rehearsals	Demo freeze and backup ready

13. Judge-Facing Proof Package

The final project folder should contain:

1. A one-page problem and solution summary.
2. The official requirement matrix.
3. A system architecture diagram showing the Java/OOP core.
4. A class or interface diagram showing Factory, Strategy, Observer, Adapter, and `SatelliteTask` .
5. A screenshot of the query interface.
6. A screenshot of the result and evidence overlay.
7. A screenshot of the execution trace.
8. A change-analysis example.
9. An optical–SAR example, with sensor-specific evidence.
10. Held-out evaluation results and baseline comparison.
11. Failure-case results and limitations.
12. Clean setup instructions.
13. Bundled sample imagery.
14. A recorded five-minute demo.
15. The final six-slide SIH presentation.

14. Six-Slide Presentation Story

The final presentation should contain exactly six slides:

Slide	Purpose	Main visual
Cover	Identity, PS ID, team, theme	Institutional title treatment

Challenge and solution	Show bottleneck and SatQuery response	Problem-to-solution scenario map
System architecture	Prove technical maturity	Java/OOP system diagram
Multimodal workflows and UI	Prove real workflows	Change pipeline, fusion pipeline, dashboard mockup
Feasibility and team strength	Prove execution realism	Java/OOP timeline and risk decision flow
Impact and research evidence	Close with value and credibility	Traditional-vs-SatQuery comparison and references

The presentation must not claim Spring Boot, Python ownership, measured accuracy, confidence, latency, or model training unless the team has actually implemented and measured them.

15. Demo Script

Opening: “Satellite imagery contains answers, but the path from image to answer is still difficult for non-GIS users. SatQuery AI makes that path askable and explainable.”

Input:

Upload an image pair and ask, “What changed in this area between the two dates?”

Trace: Point to the Java validation step, the selected `ChangeHandler`, the selected `ChangeStrategy`, the engine response, and the evidence combiner.

Result: Show the change overlay, plain-language answer, limitations, and report export.

Java strength: Explain that the controller, interfaces, patterns, trace, error handling, and report flow are Core Java/OOP responsibilities. The AI engine is replaceable behind a stable adapter.

Second proof: Show the optical–SAR workflow and explain what each sensor contributes.

Close: “SatQuery AI does not hide analysis behind a black box. It asks, routes, shows evidence, and preserves the path to the answer.”

16. Final Submission Checklist

Product

- ☐ Official requirements copied and mapped.

- ☐ One complete vertical slice works.
- ☐ Java controller and interfaces are tested.
- ☐ Specialist handlers have clear contracts.
- ☐ Trace is visible.
- ☐ Evidence and limitations are visible.
- ☐ PDF and JSON reports export correctly.
- ☐ Invalid inputs produce useful messages.

Evaluation

- ☐ Test library contains 15–20 representative queries.
- ☐ Test set is held out.
- ☐ Baseline is documented.
- ☐ Results use real measured values only.
- ☐ Failure cases are recorded.
- ☐ Unsupported claims have been removed.

Presentation

- ☐ Exactly six slides.
- ☐ Architecture diagram is readable.
- ☐ Change and fusion workflows are visible.
- ☐ Java/OOP strength is clearly stated.
- ☐ No Spring Boot or Python requirement is implied.
- ☐ Team names and IDs are correct.
- ☐ Live demo is rehearsed.
- ☐ Backup video and bundled data are ready.

Team readiness

- ☐ Every member knows the complete workflow.
- ☐ Every member can explain their owned module.
- ☐ The team can explain Factory, Strategy, Observer, and Adapter patterns.

- ☐ The team can state limitations honestly.
 - ☐ One member can recover the demo if another member is unavailable.
-

Final Recommendation

Build the **Java/OOP backbone first**. Do not make Python, Spring Boot, advanced fine-tuning, or a complex infrastructure stack a prerequisite for success. Your strongest and most defensible story is:

SatQuery AI combines specialized satellite analysis with a clean Core Java/OOP orchestration layer that makes every task modular, testable, explainable, and replaceable.

This is a technically credible project strategy because it aligns the system architecture with what your team actually knows. It also gives judges a clear reason to trust that the project can be completed and demonstrated.

References

- [1] Smart India Hackathon, official problem statement and submission instructions: use the exact official SIH 2026 source supplied by the organizers.
 - [2] SatQuery AI project execution plan supplied by the team, revised for Core Java/OOP implementation.
 - [3] GeoChat / remote-sensing vision-language model research, to be cited with the exact paper or repository used by the team.
 - [4] BigEarthNet and Sentinel-1/Sentinel-2 dataset documentation, to be cited with the exact dataset source used by the team.
 - [5] VRSBench, RSVQA, and CDVQA benchmark documentation, to be cited with the exact sources used during evaluation.
-

SIH Readiness Upgrade — Mandatory Improvements

This section converts the SIH-style rating feedback into binding project requirements. These items are not optional enhancements. They must be completed, evidenced, and checked before the team claims that the project is submission-ready.

A. Mandatory remote-sensing adaptation

The official problem statement requires at least one visual or vision-language component to be fine-tuned or otherwise adapted using BigEarthNet.txt or approved open-source training data. The project may begin with a baseline-first strategy, but adaptation must become a mandatory final milestone.

Adaptation acceptance checklist

Requirement	Required evidence	Owner	Status
Adapted component selected	Model/component name, version, and task	AI integration owner	Not started
Dataset identified	BigEarthNet subset or approved open-source source	Data owner	Not started
Adaptation method documented	Fine-tuning, LoRA/adapter, classifier head, prompt/domain adaptation, or other defensible method	AI integration owner	Not started
Data split documented	Development, validation, and held-out test partitions	Data owner	Not started
Before/after comparison	Baseline versus adapted component on held-out cases	Evaluation owner	Not started
Reproducibility package	Configuration, sample count, preprocessing, and run instructions	Packaging owner	Not started
Java integration	Adapted component accessible through <code>ModelClient</code> or a Java-compatible runtime	Java owner	Not started

Do not claim that the model was trained from scratch unless the team actually performed that work. A smaller, reproducible adaptation is stronger than an unsupported claim about a large model.

B. Official requirement-to-proof matrix

The team must replace the placeholders below with the exact wording from the official problem statement. Every row must contain an implementation artifact, a test query, a screenshot or output, a failure case, and a responsible owner.

Official requirement	SatQuery implementation	Test query	Proof artifact	Failure case	Owner	Status
Remote-sensing adaptation	Adapted VLM/component	Adaptation test query	Before/after result	Unadapted baseline	Name	No
Single-image VQA	VqaHandler	“Describe the land-cover...”	Answer screenshot	Invalid image	Name	No
Captioning or grounding	GroundingHandler or CaptionHandler	“Where are the buildings?”	Overlay/caption	No target found	Name	No
Bi-temporal change	ChangeHandler + ChangeStrategy	“What changed?”	Change map + explanation	Misaligned pair	Name	No
Optical–SAR analysis	FusionHandler + sensor evidence	“Use both sensors...”	Optical/SAR/joint output	Missing modality	Name	No
Agentic orchestration	AgentController + HandlerFactory	Mixed query set	Execution trace	Wrong route	Name	No
GUI/reporting	Upload, result, trace, PDF/JSON	Export request	UI and report	Timeout	Name	No

Gate: A requirement is not complete until a judge can see its proof in the application or in the evidence package.

C. Optical–SAR proof protocol

The team must prove that both modalities contribute to the result. A label saying “fusion” is insufficient.

For one prepared optical–SAR example, the application must show:

1. The optical input and the information extracted from it, such as visual or spectral context.
2. The SAR input and the information extracted from it, such as structural or radar evidence.
3. The combination step performed by `FusionStrategy` , `FusionHandler` , or the selected engine.
4. The joint answer and the supporting evidence.
5. A limitation message when the two modalities disagree or one modality is weak.

If true learned joint fusion is not stable, use a transparent dual-branch evidence-combination workflow and describe it accurately. Do not claim a trained joint fusion model when the system is actually combining separate outputs.

D. Change-analysis explanation generator

The change workflow must generate a natural-language explanation, not only a visual difference map. For every valid bi-temporal example, the result should state:

Explanation element	Required content
Location	Where the change is visible or the region affected
Direction	Increased, decreased, appeared, disappeared, or remained unchanged
Time	The two acquisition dates
Type	Built-up, water, vegetation, road, or another supported category
Evidence	Change map, highlighted area, or supporting image regions
Confidence	A measured value or an explicit “not available/needs review” state
Limitation	Registration quality, cloud cover, sensor difference, or unsupported inference

The Java `ChangeHandler` must return a structured `TaskResult` that contains both the visualization and this explanation.

E. Measurable evaluation protocol

Create a held-out evaluation library of 15–20 representative queries, with at least three examples for each mandatory capability where the data permits. The test files must not be used for prompt tuning, threshold selection, or model selection.

Required evaluation record

Field	Description
Query ID	Unique test identifier
Input type	Single image, bi-temporal pair, or optical–SAR pair
Expected task	VQA, grounding, change, fusion, or report
Expected evidence	Answer characteristics, region, label, or map
Actual task	Handler selected by the controller
Actual output	Answer, visualization, and metadata
Routing result	Correct or incorrect
Evidence result	Present, partial, or missing
Answer result	Correct, partially correct, incorrect, or not evaluated
Response observation	Measured latency or unavailable
Confidence	Measured confidence or unavailable
Failure category	Wrong routing, wrong answer, missing evidence, input error, slow response, or false confidence

Minimum reported metrics

Report only values that were actually measured:

- Task-routing accuracy.

- VQA answer accuracy or a documented qualitative scoring method.
- Grounding/overlay correctness where labels or masks exist.
- Change-analysis correctness and map quality where reference masks exist.
- Optical–SAR task success rate and evidence completeness.
- Report-generation success rate.
- Failure counts by category.
- Response-time observations with hardware and input size.

If a metric is not available, write **“not measured”** rather than inserting a number.

F. User validation

Before final packaging, ask at least three intended users or domain mentors to perform the primary workflow without coaching. Record:

Observation	Required record
Time to first successful query	Measured time or not measured
Confusing interface element	Exact screen or action
Misunderstood output	User interpretation and required correction
Usefulness of trace	Whether the user understood the selected task
Trust in evidence	What made the result credible or unclear
Recommended change	Concrete UI or explanation update

Use the findings to improve the upload flow, query examples, result wording, trace labels, and limitation messages. Do not claim that the system is user-friendly without this check.

G. Java-only implementation boundary

The team’s technical presentation and implementation plan must use the following boundary:

Plain Text
Core Java/OOP owns: Domain objects, validation, controller, routing, design patterns,

task interfaces, error handling, trace events, persistence, report generation, testing, and application integration.

Replaceable AI boundary owns:

Remote-sensing model inference or Java-compatible inference.

Java communicates through:

ModelClient request/response contract.

The team must not make Spring Boot or Python a prerequisite for completion. If an external service is used, document its endpoint, request schema, response schema, timeout behavior, and fallback behavior. If a Java-compatible runtime is used, record the runtime version and supported model format.

H. Evidence-first final gate

The project is ready for SIH demonstration only when all of the following evidence exists in one folder:

- Official problem statement and completed requirement matrix.
- Adaptation record with before/after held-out results.
- Java/OOP class and architecture diagrams.
- Unit-test and integration-test results.
- Single-image VQA output.
- Captioning or grounding output.
- Bi-temporal change map and natural-language explanation.
- Optical–SAR output showing both sensor contributions.
- Agentic execution trace.
- PDF and JSON report.
- Evaluation table with honest metrics and limitations.
- User-validation notes.
- Failure-case log.
- Clean setup instructions.
- Bundled sample data.
- Six-slide presentation.
- Five-minute recorded fallback demo.

I. Updated winning-readiness checklist

Readiness level	Conditions
Not ready	Plan only, no vertical slice or evidence
Prototype ready	Core Java workflow and one handler work locally
Internal demo ready	All mandatory paths exist, but evaluation or adaptation evidence is incomplete
SIH submission ready	Mandatory adaptation, five functional areas, trace, reports, held-out results, and clean demo are complete
Top-tier ready	Submission-ready plus user validation, polished UI, clear limitations, strong rehearsals, and evidence that judges can inspect quickly

J. Final non-negotiable rules

1. Adapt at least one remote-sensing visual or vision-language component before final submission.
2. Demonstrate every official requirement with an actual artifact.
3. Prove optical–SAR contribution instead of merely naming fusion.
4. Generate a natural-language change explanation, not only a map.
5. Use held-out evaluation and a baseline comparison.
6. Remove every unsupported accuracy, confidence, speed, or impact number.
7. Keep the Java/OOP boundary honest and defensible.
8. Rehearse from a clean environment and maintain a local fallback.
9. Make limitations visible instead of hiding uncertain outputs.
10. Freeze scope when the complete demo is reliable.

Final success condition: SatQuery AI is not complete when the architecture is written. It is complete when the team can run the Java-centered workflow, demonstrate every mandatory capability, show adaptation and evaluation evidence, explain optical–SAR

and change results honestly, and recover gracefully when an AI engine or network connection fails.

Final Technical Corrections for SIH Submission

1. Selected Java implementation path

To avoid ambiguity during judging, the team will use this primary path:

Plain Text

```
Browser UI: HTML + CSS + small JavaScript layer
      ↓ HTTP / JSON
Core Java HttpServer
      ↓
AgentController
      ↓
HandlerFactory → SatelliteTask → Strategy
      ↓
ModelClient → adapted model endpoint or Java-compatible runtime
      ↓
TaskResult → EvidenceCombiner → TraceObserver → ReportBuilder
      ↓
PDF / JSON report + query history
```

This path requires no Spring Boot. The application backend is a Core Java HTTP server using the Java standard library or another simple Java component that the team can fully understand and defend. The AI engine is external or replaceable; Java owns the application workflow and integration contract.

2. Java class and interface map

Plain Text

```
AgentController
  |-- uses HandlerFactory
  |-- publishes TraceEvent
  |-- returns TaskResult

HandlerFactory
```

```

|-- creates VqaHandler
|-- creates GroundingHandler
|-- creates ChangeHandler
|-- creates FusionHandler

SatelliteTask
|-- getTaskType()
|-- validate(QueryRequest)
|-- execute(QueryRequest)

TaskStrategy
|-- VqaStrategy
|-- GroundingStrategy
|-- ChangeStrategy
|-- FusionStrategy

ModelClient
|-- sends structured JSON request
|-- parses structured JSON response
|-- maps timeout/error states

TaskResult
|-- answer
|-- evidence
|-- confidence state
|-- limitations
|-- metadata

TraceObserver ← TraceEvent
EvidenceCombiner → ReportBuilder → PDF / JSON

```

The team must prepare unit tests for Factory creation, Strategy selection, invalid modality handling, missing-pair handling, failed model response handling, and trace-event ordering.

3. Team-owned versus replaceable responsibilities

Team owns in Core Java/OOP	Replaceable or external engine
Request and domain classes	Remote-sensing model inference
File and modality validation	Adapted VQA or fusion model
AgentController	GPU execution, if applicable
Factory and Strategy routing	Model endpoint availability
Observer trace events	Model-specific computation

EvidenceCombiner	Specialized image model
ReportBuilder	External dataset service
Unit and integration tests	Adaptation training environment

4. Updated mandatory project gates

Gate	Deadline	Pass condition
G0: Official requirements	Day 1	Exact five requirements copied and assigned
G1: Java foundation	Week 1	Domain objects, interfaces, controller, mock task, and tests run locally
G2: Adaptation decision	Week 1	One adaptation route selected; data, method, owner, and evidence defined
G3: Evaluation foundation	Week 2	Held-out test library, baseline, and scoring format work
G4: Vertical slice	Week 3	UI → Java controller → handler → result → trace works
G5: Adaptation proof	Week 4–5	Adapted component, before/after held-out comparison, and Java integration complete
G6: Multimodal proof	Week 5	Change explanation and optical–SAR contribution are visible
G7: Demo freeze	Week 6	Reports, error handling, local fallback, rehearsal, and six-slide deck complete

5. Final claim-control rules

The team must not claim an accuracy, confidence, latency, percentage improvement, model-training result, enterprise readiness, or user benefit unless it has been measured or

documented. Every number in the presentation and report must have a source, sample count, evaluation method, and owner.

Use “not measured,” “baseline demonstration,” “needs review,” or “planned” where evidence does not yet exist. Honest limitations increase credibility and protect the team during judge questions.

6. Final SIH readiness condition

SatQuery AI can be called SIH-ready only when the team has completed all of the following:

- One remote-sensing component adapted using BigEarthNet.txt or approved open-source data.
- Before/after held-out evaluation of that adaptation.
- A Core Java vertical slice with working Factory, Strategy, Observer, and Adapter responsibilities.
- Single-image VQA and one additional single-image task.
- Bi-temporal change analysis with natural-language explanation.
- Optical–SAR analysis that visibly proves both modalities contributed.
- Agentic task and tool selection with an auditable trace.
- Evidence, limitations, and confidence state shown to the user.
- PDF and JSON report export.
- Failure-case handling and local demo fallback.
- User validation with recorded findings.
- Official requirement-to-proof matrix completed.
- Clean setup instructions and a rehearsed live demonstration.

Final positioning: SatQuery AI is a Core Java/OOP application that turns natural-language satellite questions into validated, routed, evidence-backed analysis through replaceable remote-sensing AI engines. The team owns the reliable software system; the AI engines remain cleanly integrated modules.